



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

Biometría: Filtros, Curvas ROC y Detector facial Viola Jones

TRABAJO TECNOLOGÍAS DEL LENGUAJE HUMANO

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital

Gómez Corral, Miquel

CAPÍTULO 1

Introducción

En este trabajo se ha realizado la implementación de tres de los ejercicios propuestos para la asignatura de Biometría: Filtros faciales, cálculos relacionados con la curva ROC y el detector facial Viola Jones.

De filtros faciales, se han implementado la normalización global y local por media y desviación típica únicamente. No se ha pretendido implementarlas todas pero sí las más relevantes.

Con respecto a la curva ROC, se ha implementado en un *notebook* todo el cálculo de falsos positivos (FP) y falsos negativos (FN) dado un umbral, así como el FN para un FP y viceversa. Hay visualizaciones de la curva ROC y el cálculo AUC.

Para la recreación del detector facial Viola Jones, se han implementado todas las partes mencionadas en el *paper* original: extracción de características Haar, selección de características con AdaBoost, clasificación en cascada, *hard negative mining* y varias optimizaciones adicionales. Una demo en tiempo real del detector puede verse en <https://youtu.be/JnWS4aAlpVo>.

Hardware utilizado

Los experimentos se han realizado en mi máquina personal: GPU NVIDIA RTX 5070 (12 GB VRAM), procesador AMD Ryzen 9 9000X, 64 GB de RAM y Ubuntu 24.04 LTS.

Sin embargo, dado el alto coste espacial de Viola Jones, se ha optado por realizar el experimento grande en el clúster Tensor, usando 250 GB de RAM y 32 CPUs. Los experimentos pequeños con unas 15k muestras se han podido realizar en la máquina personal sin problemas; los demás, en el clúster.

1.1 Código entregado

- `notebooks/Ej1-Filtres.ipynb`: Implementa todo lo relacionado con los filtros faciales.
- `notebooks/Ej2-CurvaRoc.ipynb`: Implementa todo lo relacionado con la curva ROC.
- `notebooks/Ej3-TrainViolaJones.ipynb`: Muestra todos los pasos necesarios para cargar los datos, extraer las características Haar, seleccionar las mejores características con AdaBoost y entrenar el clasificador en cascada para un reconocimiento facial basado en el *paper* de Viola y Jones.
- Repositorio y otros notebooks: El repositorio cuenta con varios scripts para el desarrollo. A través de `/app/main.py` se pueden lanzar las dos funciones principales: activar la cámara para usar el detector y otra para entrenar una instancia del modelo. Cuenta con más notebooks para la visualización de diferentes aspectos del proyecto en `/notebooks`. Más detalles sobre cómo usar el repositorio en el `README.md` incluido en el mismo.

CAPÍTULO 2

Filtros

Mencionar en esta sección, que en el contexto de la detección facial, el preprocesamiento de las imágenes es fundamental para garantizar la robustez del sistema frente a variaciones de iluminación y contraste. En este trabajo se ha implementado un filtro de normalización por media y desviación estándar local.

La normalización local se define, para cada píxel (i, j) , como:

$$I_{\text{norm}}(i, j) = \frac{I(i, j) - \mu_W(i, j)}{\sigma_W(i, j) + \epsilon}, \quad (2.1)$$

donde $\mu_W(i, j)$ y $\sigma_W(i, j)$ son la media y la desviación estándar calculadas en una ventana W centrada en el píxel, y ϵ es una constante de regularización para evitar divisiones por cero.

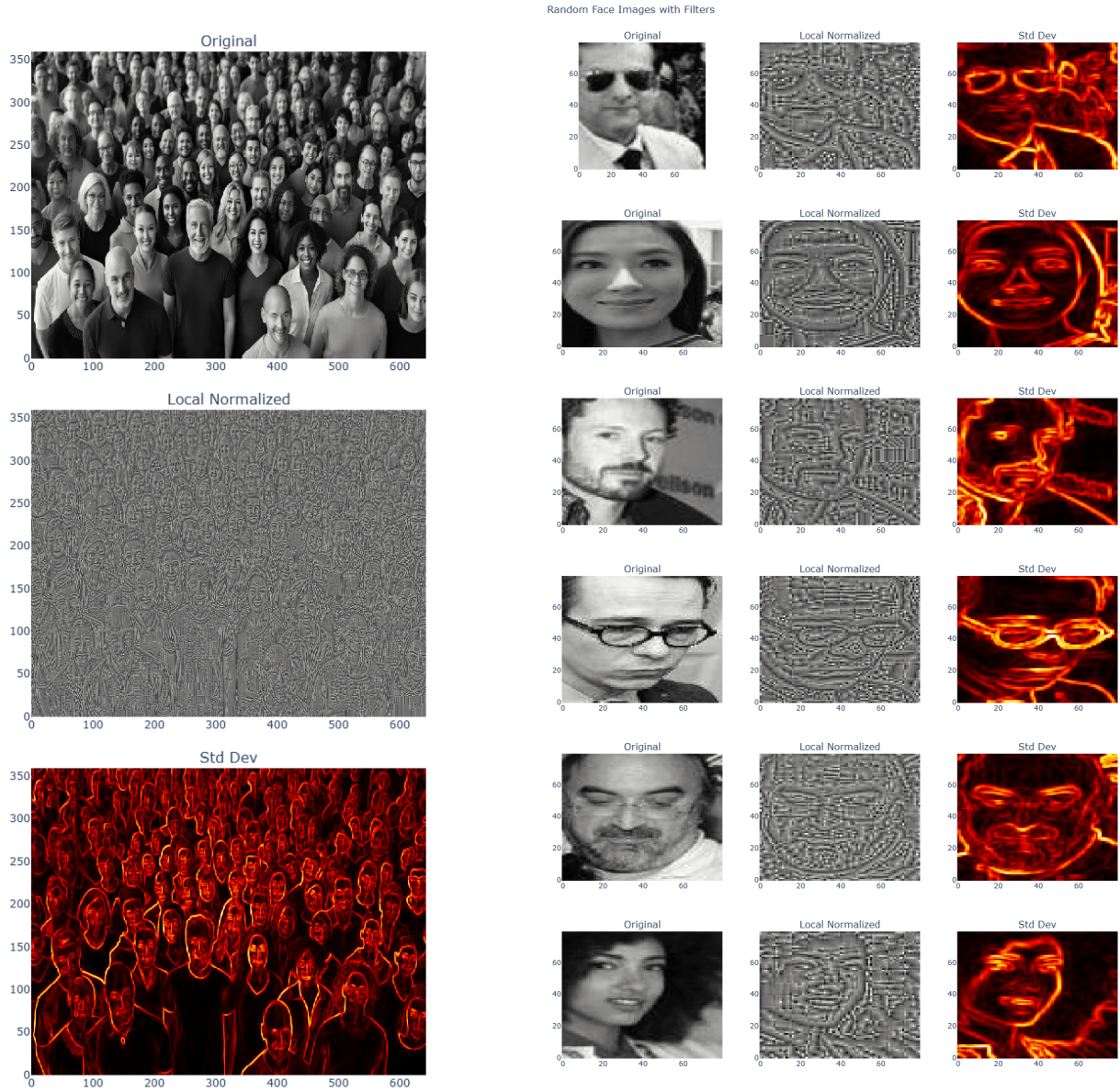
Para hacer este cálculo de las medias y la desviación estándar computacionalmente eficiente, se utilizan *imágenes integrales*. A partir de la imagen original I , se construye la imagen integral S y la imagen integral de cuadrados S_2 , de modo que la suma (y la suma de cuadrados) de cualquier región rectangular puede obtenerse en tiempo constante con solo cuatro accesos a memoria. Esto permite calcular la desviación estándar local para todos los píxeles de forma vectorizada, sin necesidad de recorrer explícitamente cada ventana.

En la Figura 2.1 se muestran los resultados de aplicar este filtro sobre dos tipos de imágenes: una fotografía de multitud (vista general) y varios recortes de caras individuales. En ambos casos se presentan tres vistas: la **imagen original** en escala de grises, la **versión localmente normalizada** y el **mapa de desviación estándar local** (donde los valores altos, en tonos cálidos, indican mayor variabilidad textural, típicamente asociada a bordes y contornos).

Como se observa en la Figura 2.1a, la normalización local resalta los contornos faciales incluso en condiciones de iluminación desuniforme, mientras que el mapa de desviación estándar evidencia las regiones de mayor contraste, como los bordes de las caras y los rasgos faciales.

En el caso de las caras individuales de la Figura 2.1b, el efecto es más pronunciado: la imagen normalizada reduce las diferencias en bruto entre muestras, y el mapa de desviación estándar destaca de manera consistente los ojos, la nariz, la boca y el contorno de la cara, lo cual resulta especialmente útil para la extracción posterior de características Haar.

Nota: la imagen normalizada puede parecer ruidosa a baja resolución; se recomienda ampliar el PDF para apreciar los detalles.



(a) Imagen de multitud con sus correspondientes filtros. De arriba abajo: original, normalizada localmente y desviación estándar.

(b) Malla de caras individuales mostrando el efecto de los filtros. Cada fila presenta, de izquierda a derecha: original, normalizada localmente y desviación estándar.

Figura 2.1: Resultados de la normalización local y el cálculo de desviación estándar sobre imágenes de multitud (izquierda) y caras individuales (derecha).

CAPÍTULO 3

Curva ROC

La curva ROC es una herramienta para evaluar un sistema de clasificación, permite visualizar el compromiso entre TPR, *True Positive Rate* y la FPR, *False Positive Rate* al variar el umbral de decisión del clasificador.

En este trabajo se ha implementado un *notebook* que, dado un conjunto de puntuaciones de clientes e impostores, calcula estos ratios de forma exhaustiva para todos los umbrales posibles.

Se han utilizado dos conjuntos de datos, denominados A y B, cada uno con 2 990 muestras (clientes e impostores mezclados). Las puntuaciones representan la confianza del sistema de que una muestra pertenece a un cliente legítimo. Para un umbral dado θ , se definen:

- **Falsos Positivos (FP):** impostores clasificados erróneamente como clientes ($\text{score} > \theta$).
- **Falsos Negativos (FN):** clientes rechazados erróneamente ($\text{score} \leq \theta$).

A partir de estos valores se computan las tasas FPR y FNR para cada umbral, permitiendo trazar la curva ROC ($\text{TPR} = 1 - \text{FNR}$ frente a FPR). Además, se calcula el área bajo la curva (AUC) mediante integración numérica por el método del trapecio (que se calcula en tiempo lineal, en lugar de cuadrático comparando todos los clientes con los impostores como hacemos en clase con pocos datos), obteniendo una métrica global de la capacidad discriminativa del sistema.

Métrica	Dataset A	Dataset B	Umbrales (θ_A / θ_B)
AUC	0,8831	0,8830	—
D prime	0.7597	0.8732	—
FN = FP	20,14 %	20,07 %	0,0503 0,0444
FP para FN = 1 %	80,13 %	68,00 %	0,0071 0,0000
FP para FN = 5 %	57,56 %	53,65 %	0,0202 0,0048
FP para FN = 20 %	20,45 %	20,13 %	0,0500 0,0443
FN para FP = 1 %	62,73 %	64,55 %	0,1474 0,2999
FN para FP = 5 %	39,58 %	44,55 %	0,0826 0,1444
FN para FP = 20 %	20,14 %	20,42 %	0,0504 0,0450

Tabla 3.1: Resultados principales del análisis ROC para los datasets A y B.

Los resultados, resumidos en la Tabla 3.1, muestran que ambos datasets presentan un comportamiento muy similar, con un AUC cercano a 0,88, lo que indica una buena capacidad de discriminación del sistema. No obstante, el punto de igual error (donde $\text{FN} \approx \text{FP}$) se sitúa en torno al 20 %, lo cual

sugiere que, para aplicaciones biométricas exigentes, sería necesario ajustar el umbral en función de si se prioriza minimizar los falsos positivos o los falsos negativos.

Por ejemplo, si se fija un umbral de $\theta = 0,10$, el dataset A presenta una tasa de FP del 2,63 % y una tasa de FN del 48,04 %, mientras que el dataset B arroja valores de 8,53 % y 34,62 %, respectivamente. Estas diferencias evidencian que la distribución de puntuaciones varía entre conjuntos de datos, por lo que la elección del umbral debe adaptarse al escenario de aplicación concreto.

Para obtener una tasa de FN en torno al 1 % (apenas rechazar clientes) es necesario reducir drásticamente el umbral. En el dataset A se requiere un umbral de 0,0071, mientras que en el dataset B ni siquiera existe un umbral positivo que consiga ese FN, por lo que el valor más cercano es $\theta = 0,0000$, equivalente a aceptar toda puntuación mayor que cero. Ni siquiera con $\theta = 0$ se alcanza un FP del 100 %, ya que aproximadamente un 31 % de los impostores tiene puntuación exactamente cero y, por tanto, son correctamente rechazados.

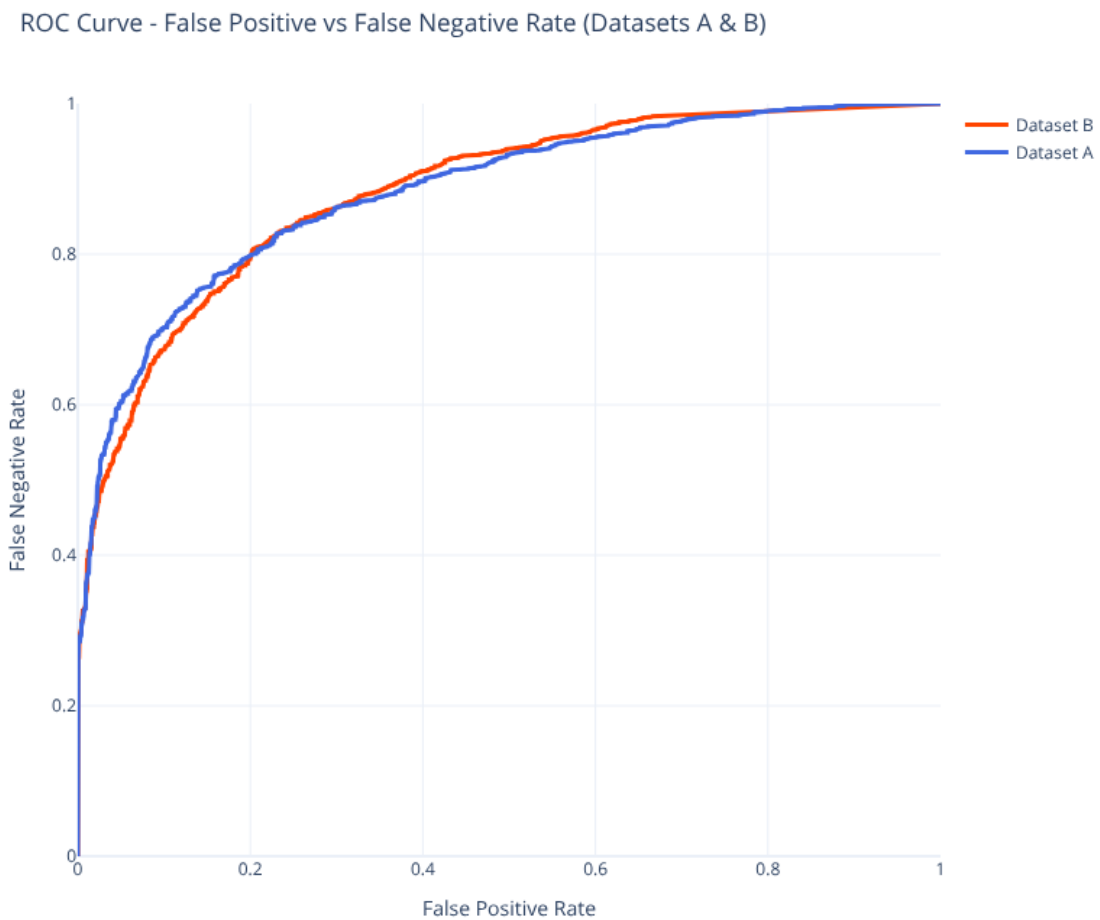


Figura 3.1: Curva ROC para los datasets A y B.

Ambos datasets muestran curvas prácticamente solapadas, lo que confirma que el sistema de puntuación se comporta de forma similar en ambos conjuntos. El AUC cercano a 0,88 indica una capacidad de discriminación aceptable, aunque la zona de bajo FP revela una caída pronunciada de la TPR, reflejando la dificultad de mantener una alta tasa de acierto cuando se exige una tasa de falsos positivos muy reducida.

CAPÍTULO 4

Viola Jones

Para la implementación de un detector facial basado en el método de Viola y Jones, se ha seguido de cerca la descripción del *paper* original, adaptándola a las herramientas y librerías actuales. A continuación se describen los pasos realizados:

1. **Obtención de datos:** Se nos ha provisto de un *dataset* de imágenes de caras, pero no de no-caras. Por lo tanto, primero se ha tenido que encontrar un *dataset* sin caras del que poder extraer las muestras de *hard mined*.
2. **Listado de características Haar:** Para poder clasificar usando predictores débiles, se han de listar todas las posibles características de las imágenes, las cuales se obtienen aplicando filtros Haar. Por recomendación del profesor, y para evitar saturar la memoria, se ha reducido el conjunto de características, en este caso, limitando el tamaño mínimo de los filtros.
3. **Selección de características:** La implementación requiere crear diferentes etapas para poder entrenar el clasificador en cascada. Para cada etapa, se ha utilizado el algoritmo AdaBoost para seleccionar las mejores características de entre las listadas. Con los n mejores filtros seleccionados en esa etapa, se ha ajustado el umbral para maximizar la tasa de verdaderos positivos (TPR) y minimizar la tasa de falsos positivos (FPR).
4. **Hard mining:** Para cada etapa, se han extraído las muestras de *hard mining* del *dataset* sin caras, es decir, aquellos *crops* de imágenes que el clasificador (formado por las etapas previas a la actual) ha clasificado erróneamente como caras. Estas muestras se han añadido al conjunto de entrenamiento para la siguiente etapa con el objetivo de que, en cada etapa, el clasificador AdaBoost tenga una cantidad balanceada de muestras de cada clase.
5. **Clasificador en cascada:** Finalmente, se ha implementado el clasificador en cascada, que consiste en encadenar las diferentes etapas entrenadas. De esta forma, una imagen se clasifica como cara si y solo si pasa todas las etapas del clasificador.

4.1 Obtención de datos

Para el entrenamiento del modelo, se requiere un conjunto de imágenes con caras (muestras positivas) y otro sin ellas (muestras negativas). Mientras que las muestras positivas se procesan y precalculan una única vez al inicio del entrenamiento, las muestras negativas se obtienen de forma dinámica durante el proceso.

4.1.1. Dataset de NO caras

Para las muestras negativas, se extraen muestras del conjunto de datos open-images-v7 empleando la librería `fiftyone`.

Dado que este conjunto de datos es general y contiene imágenes con caras, se ha usado la siguiente estrategia para seleccionar un conjunto de imágenes sin caras:

- Este dataset viene anotado con muchas etiquetas para 'clasificación' multi-etiqueta. Lo primero que se ha hecho es listar una gran cantidad de estas etiquetas y seleccionar aquellas que, casi en su totalidad, garantizan la ausencia de caras en las imágenes asociadas. Ejemplos son: *Ant*, *Apple*, *Artichoke*, *Bagel*, *Baked goods*, *Banana*, *Bee*, *Beetle*. Como son multi etiqueta, se excluyen aquellas que puedan incluir imágenes con caras, como por ejemplo: *Shirt*, *Human arm*, *Human face*, *Blond girl*... En la Figura 4.1 se muestran ejemplos de imágenes obtenidas mediante este filtrado:



Figura 4.1: Ejemplos de imágenes sin caras extraídas del dataset Open Images V7 filtrado por etiquetas

- Aún así, estas imágenes pueden contener caras de forma ocasional, ya que incluir o excluir ciertas etiquetas no garantiza la ausencia de caras. Por lo tanto, ha sido necesario un filtrado final en el que se usa el detector de caras de OpenCV para eliminar aquellas que contengan caras. El uso de esta herramienta resulta algo contradictorio, ya que se basa en el mismo método de Viola Jones. Ahora bien, como este es un trabajo académico, se ha optado por esta solución para garantizar la calidad de los datos, aunque en un entorno real se podrían usar técnicas de detección facial más modernas para este filtrado.
- En total, se han obtenido más de 128 mil imágenes sin caras, lo que ha permitido generar un gran número de muestras negativas para el entrenamiento del modelo.

Se ha dividido todo el *dataset* en dos particiones: 10 % para test y el resto para generar las muestras de entrenamiento. Luego, con ese 10 % de test, se han generado 2,5 millones de *crops* sin caras estáticos (sin regenerar), que se han usado para ir controlando la tasa de falsos positivos a medida que se van añadiendo nuevas etapas al clasificador en cascada. De cada imagen, se han extraído $2,5\text{ M}/12,8\text{ K} \approx 195$ *crops*.

4.1.2. Dataset de caras

Para los datos positivos, se intentó utilizar las imágenes de caras proporcionadas en el *dataset* inicial. El problema es que estas imágenes contienen caras pero no centradas en los ojos y la nariz, como se requiere para el entrenamiento del modelo. Al no ser un modelo neuronal, no es capaz de aprender a detectar las caras a partir de imágenes con posiciones y tamaños muy variados, sino que necesita un formato específico para poder extraer las características Haar de forma efectiva.

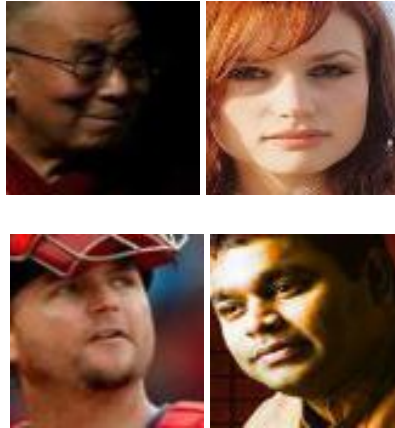


Figura 4.2: Ejemplos de caras extraídas del dataset.

A continuación, en la figura, se muestra la 'cara media' que se obtiene al superponer todas las imágenes de caras del *dataset*. Luego, la versión limpia que conseguimos tras procesar este dataset.

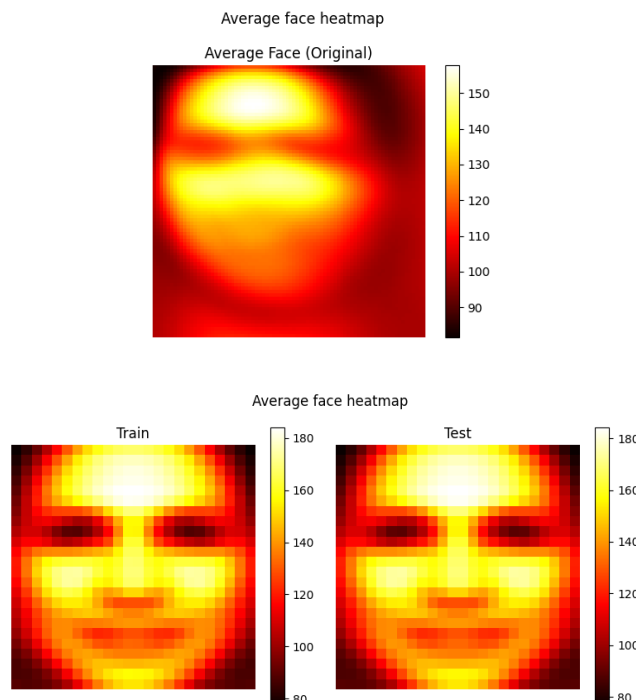


Figura 4.3: Cara media obtenida al superponer todas las imágenes de caras del dataset. La imagen superior corresponde a la cara media obtenida con las imágenes originales, mientras que la imagen inferior corresponde a la cara media obtenida con los *crops* centrados en los ojos y la nariz y reescaladas a 24×24 píxeles.

Con esto, se evidencia que las caras no están centradas ni alineadas, sino que hay una gran variedad de posiciones y tamaños (figura 4.3): muestra una mancha borrosa.

Por lo tanto, se ha optado por procesar este *dataset* usando, de nuevo, el detector de caras de OpenCV. Se extraen *crops* de 24×24 píxeles para los que el modelo tenga una confianza alta de que son caras (el detector devuelve un valor de confianza entre 0 y 20, siendo lo normal 5; pero aquí lo subimos: si supera 8, se conservaba la imagen). De esta forma, se garantiza prácticamente que estén centrados en los ojos y la nariz. Con esta técnica, se pasa de tener más de 75k caras a tener solo más de 26k. Si se genera su versión espejo, se obtienen más de 52k. Se ha reservado un 10% de estas muestras para test, y el resto para entrenamiento.

Ahora, si se visualiza la 'cara media' con los *crops* centrados figura 4.3, se puede observar que sí hay una cara clara, lo que evidencia que ahora sí están centradas y alineadas.

4.2 Listado de características Haar

Para la extracción de características, se recorren de forma iterativa todas las posiciones y tamaños posibles de los filtros Haar sobre los *crops* de 24×24 píxeles.

Durante este listado, se **omite el cálculo de los filtros invertidos** (por ejemplo, el patrón blanco-negro frente a negro-blanco), ya que el propio algoritmo AdaBoost se encarga de determinar el signo adecuado del peso asociado, lo cual reduce el espacio de características a la mitad.

Además, para garantizar la máxima eficiencia computacional, el cálculo del valor de todos los filtros Haar se realiza mediante la técnica de la **imagen integral**.

4.2.1. Mejoras y optimizaciones

Si se calculan todos los filtros Haar posibles en el espacio 24×24 , se llega a casi 170k. Esto supone una gran uso de memoria, y más teniendo en cuenta que se han de calcular para cada imagen.

Para evitar esto, se han aplicado dos optimizaciones:

- Precalcular todas las características de las caras antes de entrenar (son estáticas).
- Limitar el tamaño mínimo de los filtros. Ya que los de pocos píxeles no parecían ser usados por el modelo, y tampoco se veían en la versión oficial de OpenCV, se han limitado a estos tamaños según el filtro:
 - Filtros horizontales: mínimo de 6×4 píxeles.
 - Filtros verticales: mínimo de 4×6 píxeles.
 - Filtros de 3 horizontales: mínimo de 9×4 píxeles.
 - Filtros de 3 verticales: mínimo de 4×9 píxeles.
 - Filtros de 2 diagonales: mínimo de 6×6 píxeles.

Con esto se pasa a tener unos 79k filtros, la mitad.

Por último, por recomendación de un compañero de clase, se ha añadido un tipo de filtro extra: un cuadrado con otro cuadrado concéntrico en su interior, de menor tamaño. Este filtro captura patrones de borde tipo marco, similares a los filtros de 3 rectángulos pero con una proporción diferente.

Limitando su tamaño mínimo a 6×6 píxeles, se añaden unos 2k filtros más, llegando a un total de unos 81k filtros Haar, lo que sigue siendo una cantidad manejable para el entrenamiento del modelo, especialmente con las optimizaciones implementadas en la extracción de características y la selección de características con AdaBoost.

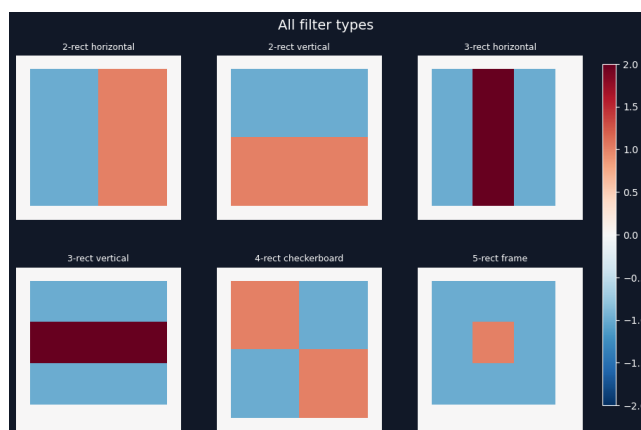


Figura 4.4: Tipos de filtros Haar utilizados en la implementación del detector facial Viola Jones. Se incluyen los filtros horizontales, verticales, de 3 horizontales, de 3 verticales, de 2 diagonales y el filtro cuadrado con otro cuadrado concéntrico.

4.3 Selección de características

Una vez evaluadas las características de una imagen, se emplea el algoritmo AdaBoost para seleccionar aquellas que mejor separan las muestras positivas de las negativas.

En este proyecto, se ha integrado una versión personalizada de AdaBoost (implementada con la ayuda de Claude), que ofrece una interfaz similar a la de `scikit-learn`, pero que está fuertemente especializada en el uso de *stumps* de decisión y paraleliza la búsqueda del mejor *split* entre todas las características. La implementación original de AdaBoost con `scikit-learn` era demasiado lenta para el número de características y muestras que se manejan, por lo que esta versión personalizada ha sido crucial para poder completar el entrenamiento en un tiempo razonable.

Luego, cada vez que se termina un *stump*, el algoritmo ajusta el umbral de la etapa para dejar pasar, como mínimo, un 99,9 % de caras ($\text{TPR} \geq 0,999$). Con ese nuevo umbral, se calcula FP, y si llega al objetivo del 50 % de FP, se detiene el entrenamiento de esa etapa y se pasa a la siguiente.

Esto consigue que, tras todas las etapas, la probabilidad de que un *crop* sin cara pase todas las etapas se reduzca significativamente. Además, al descartar los clasificadores débiles innecesarios, se reduce el coste computacional de la etapa.

Si tras un número máximo de *stumps* no se ha alcanzado el objetivo de FP, se detiene igualmente el entrenamiento de esa etapa y se pasa a la siguiente. Esto se hace para evitar que una etapa tenga un número excesivo de *stumps*, lo que podría llevar a un sobreajuste y tiempo de inferencia excesivo.

4.3.1. Mejoras

La mayoría de experimentos se han realizado con una tasa de FP objetivo del 50 % y TPR del 99,9 %, pero, por recomendación del profesor, se ha probado a ajustar el valor de las primeras etapas a valores más bajos.

En concreto, se ha llevado esta idea a unos ratios progresivos:

- Etapas 1 a 2: tasa de FP objetivo del 15 %.
- Etapas 3 a 5: tasa de FP objetivo del 25 %.
- Etapas 6 a 10: tasa de FP objetivo del 40 %.
- Etapas 11 en adelante: tasa de FP objetivo del 50 %.

Todo esto con la idea de forzar a que las primeras etapas, donde el *hard mining* no es 'tan difícil', se enfoquen en eliminar la mayor cantidad de muestras negativas posible, aunque esto implique un mayor número de *stumps*. Luego, a medida que se avanza en las etapas, el *hard mining* se vuelve más difícil, por lo que se relaja la tasa de FP objetivo para evitar que las etapas tengan un número excesivo de *stumps*.

También, se han hecho pruebas con una TPR de 0,99 y de 0,999. Este ajuste determina cuántas caras deben pasar en cada etapa. A cambio, de normal harán falta más *stumps* para alcanzar el objetivo de FP. Ahora, si se baja mucho, al final $0,99^{30} = 0,74$, mientras que $0,999^{30} = 0,97$. Como se ve, después de muchas etapas, se terminan perdiendo una gran cantidad de caras, lo que crea un compromiso entre la tasa de FP y la tasa de verdaderos positivos. Por eso, se realizan experimentos con ambos valores.

4.4 Hard mining

El entrenamiento de un clasificador en cascada necesita que, en cada una de las etapas sucesivas, el modelo se enfoque en los casos que más difíciles les resultan de discriminar.

Para alcanzar la cantidad necesaria de muestras negativas (hasta igualar el total de caras positivas para que el conjunto quede balanceado), se toman nuevas imágenes del conjunto `open-images-v7` y se procesan a través de todas las etapas ya calculadas. Cualquier *crop* que supere la cascada actual se añade como un falso positivo.

Con esto, se consigue implementar el *hard negative mining*. Este paso es crucial, ya que no sirve solo precalcular un gran número de *crops* sin caras; de hacer eso, en un par de etapas, el clasificador ya habrá aprendido a descartar la mayoría de ellos, y se quedará sin muestras negativas para seguir entrenando. En cambio, con esta técnica, cada etapa se entrena con las muestras negativas más difíciles de clasificar, lo que mejora significativamente la capacidad discriminativa del clasificador final.

Preservación de falsos positivos: Además, se ha implementado una estrategia de preservación de los falsos positivos de la etapa anterior. En concreto, los *crops* falsos positivos que sobreviven a la cascada actual, se concatenan con las nuevas muestras negativas generadas por *hard mining*, de forma que el conjunto de entrenamiento de cada etapa incluye tanto los nuevos falsos positivos como los heredados. Esto asegura que el clasificador no 'olvide' lo que tendría que haber aprendido a rechazar.

Diversidad: Para asegurar la diversidad de estos *crops*, no se eligen de un número reducido de imágenes, sino que se intentan extraer de muchas. Se pone un tope a la cantidad máxima de *crops* que se pueden extraer de cada imagen, para evitar que el modelo se sobreajuste a un número reducido de patrones negativos: máximo 10 % del total.

Con esto, se asegura la diversidad, sin limitar el minado cuando este se vuelva difícil: máximo 10 % implica que si hay menos (porque estamos en una etapa difícil), se eligen todos. Y al tener tantos miles de imágenes de donde sacar NO caras, con muy poca probabilidad nos quedaremos sin muestras.

4.5 Clasificador en cascada

Sabiendo ya cómo entrenar cada etapa, si se concatenan todas, se puede crear el clasificador en cascada final.

En el script de entrenamiento (`generate_all_stages`), se genera e incorpora cada etapa manteniendo un control estricto de la tasa global de falsos positivos, deteniendo el aprendizaje cuando se alcanzan los criterios de parada predefinidos en función de la tasa de FP objetivo: descartar todos los *crops* de test.

Para controlar la tasa de FP, se ha creado un *dataset* de 2,5 millones de *crops* sin caras (de la partición de test) y se va pasando por la cascada cada vez que se entrena una nueva etapa, para ir observando cómo evoluciona la tasa de FP a medida que se añaden nuevas etapas.

Si se alcanza el objetivo de tasa de FP, se detiene el entrenamiento, aunque se sigan cumpliendo los criterios de parada relacionados con el número de muestras restantes. Esto se hace para evitar un sobreajuste excesivo y para reducir el tiempo de entrenamiento, ya que a partir de cierto punto, añadir más etapas no mejora significativamente la capacidad discriminativa del clasificador, pero sí aumenta el tiempo de entrenamiento, la complejidad del modelo y reduce la cantidad de caras que pasan por la cascada, lo que puede ser contraproducente para la detección final.

4.6 Inferencia

Respecto a la inferencia, originalmente se implementó el clasificador de forma secuencial como prueba de concepto, pero era muy lento. Con la ayuda de Claude, se ha programado una versión vectorizada y paralelizada (ubicada en `app/src/model/cascade_clasifier.py`) que usa `numba` con `@njit(parallel=True)` o, como alternativa, `NumPy` con hilos, logrando una evaluación en tiempo real.

Esta versión se usa tanto para obtener las muestras de *hard mining* como para la detección facial final. Para esta última, se ha implementado un script que activa la cámara y muestra en tiempo real los resultados de la detección facial, dibujando un recuadro alrededor de las caras detectadas.

4.6.1. Pirámide de escalas y detección multi-escala

Para detectar caras de diferentes tamaños en la imagen, se implementa una pirámide de escalas: la imagen se reduce iterativamente por un factor de submuestreo (por defecto 0,8), y en cada escala se evalúan todas las ventanas candidatas. Las coordenadas de las detecciones se escalan de vuelta al espacio de la imagen original.

Además, se ofrece la opción de reducir la imagen de entrada (`halve_size_factor`) antes de iniciar la pirámide, lo que reduce significativamente el tiempo de inferencia a costa de una menor precisión en caras muy pequeñas: se limita el tamaño mínimo de crop.

4.6.2. Non-maximum Suppression (NMS)

Tras la evaluación de la cascada, es habitual obtener múltiples detecciones solapadas sobre la misma cara. Para consolidarlas, se aplica supresión de no-máximos mediante `cv2.groupRectangles`, agrupando rectángulos solapados (con un umbral $\varepsilon = 0,2$) y devolviendo la media de cada grupo.

4.6.3. Carga de cascadas pre-entrenadas de OpenCV

Para propósitos de prueba y validación externa, el sistema cuenta con un *parser* capaz de interpretar las cascadas pre-entrenadas oficiales de OpenCV descritas en XML, cargando e instanciando estas etapas para emplearlas directamente con la implementación propia construida en este trabajo.

4.7 Serialización y formato

Los modelos entrenados se serializan en formato XML compatible con OpenCV, lo que permite interoperabilidad con herramientas existentes. Se ha implementado tanto un serializador (`CascadeSerializer`) que guarda la cascada entrenada, como un *parser* (`HaarCascadeParser`) que carga cascadas pre-entrenadas

de OpenCV en formato XML, permitiendo usar la implementación propia con modelos ya existentes. El formato soporta tanto el formato antiguo (*weakClassifiers*) como el nuevo formato basado en árboles de OpenCV.

4.8 Optimizaciones

Ya que la cantidad de características Haar es tan grande, y el número de muestras también, se han implementado varias optimizaciones para reducir el tiempo de entrenamiento:

1. **Paralelización:** Tanto la búsqueda del mejor *split* en AdaBoost para cada característica, como la generación de los *hard negatives* y el cálculo de las características de cada *crop*, se han paralelizado, lo que reduce significativamente el tiempo de entrenamiento de cada etapa a unos 45 s por *stump*. Luego ya depende de la etapa y cuántos *stumps* tenga, pero se ha llegado a entrenar una etapa en unos 10 o 20 minutos, lo que es razonable para este tipo de modelo.
2. **Cantidad de imágenes:** Las imágenes del *dataset* open-images-v7 son de alta resolución, pero por las necesidades de diversidad en los *hard negatives*, se ha mantenido un número grande de imágenes: se cargaban 116 000 distintas.

Ya que se tenía que balancear el número de positivos y negativos para cada etapa, esto ha permitido reducir el número de negativos a procesar y, sobre todo, la memoria usada. Con unos 81 000 filtros Haar y $46\,000 \times 2 = 92\,000$ muestras (contando con los *hard negatives* generados en cada etapa), el espacio que ocupaba el *dataset* en memoria superaba los 200 GB, lo que hacía inviable el entrenamiento (había que usar el clúster Tensor). Con $15\,000 \times 2 = 30\,000$ muestras, se ha logrado un equilibrio entre la calidad del modelo y los recursos disponibles en local.
3. **Extracción vectorizada de características:** Se ha implementado el cálculo de todas las características Haar de una imagen en una sola pasada vectorizada usando `np.bincount`, lo que evita bucles en Python y acelera enormemente la fase de extracción.
4. **Normalización por varianza local:** Cada recorte de imagen se normaliza dividiendo sus características por la desviación estándar local (calculada también con imágenes integrales), lo que proporciona invarianza a cambios de iluminación.
5. **Aumentado de datos:** Se ha implementado un sistema de aumentado de datos (*data augmentation*) que aplica, con ciertas probabilidades, ruido gaussiano, variaciones de contraste, desenfoque gaussiano y cambios de brillo tanto a las imágenes de caras como a los fondos. Esto mejora la robustez del modelo frente a variaciones en las condiciones de captura.
6. **Checkpoint y reanudación:** El entrenamiento guarda el estado completo de la cascada tras cada etapa (clasificadores, umbrales, falsos positivos acumulados, etc.) en un archivo *pickle*. Si el entrenamiento se interrumpe, se reanuda automáticamente desde la última etapa completada, lo que es esencial dado el largo tiempo de entrenamiento.

CAPÍTULO 5

Conclusiones y Resultados (Detector)

5.1 Resultados

Para evaluar el rendimiento del clasificador en cascada, se han realizado cinco experimentos principales variando dos parámetros clave: la tasa mínima de verdaderos positivos (TPR) exigida en cada etapa (0,999 y 0,99) y el uso o no de una tasa de falsos positivos progresiva (pfp, *progressive false positive rate*), que reduce el objetivo de FP en las primeras etapas para forzar una eliminación más agresiva de negativos al inicio de la cascada.

Con el objetivo de comparar configuraciones y observar la convergencia del entrenamiento, los cuatro primeros experimentos se han realizado con un subconjunto de 15 000 muestras positivas (seleccionadas aleatoriamente del total de caras y las versiones espejo) y 15 000 muestras negativas generadas por *hard mining* en cada etapa. En paralelo, y asumiendo que sería la mejor opción, se entrenó el modelo (pfp_999_all_data) utilizando todas las muestras disponibles en el cluster.

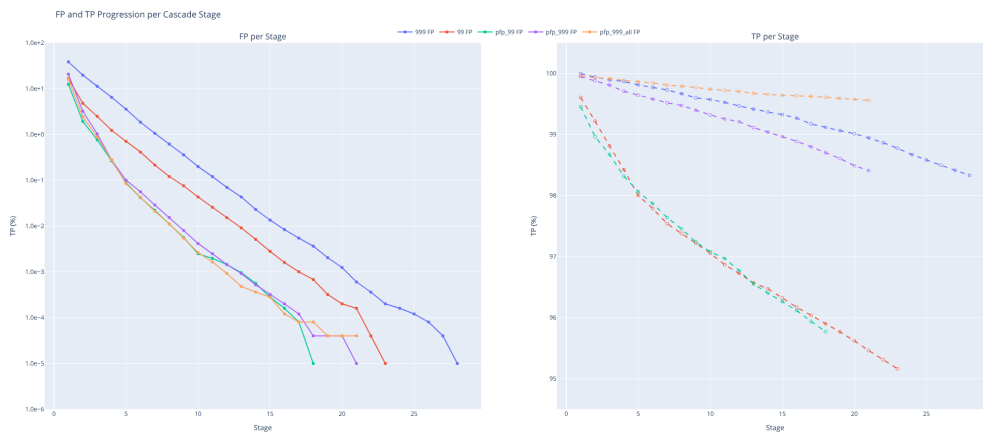


Figura 5.1: Progresión de la tasa de FP y TPR a lo largo de las etapas para las distintas configuraciones.

Modelo	Descripción	Etapas	TPR final
balanced_999	$\text{TPR} \geq 0,999$, FP fijo	28	0,9833
balanced_99	$\text{TPR} \geq 0,99$, FP fijo	23	0,9516
pfp_99	$\text{TPR} \geq 0,99$, FP progresivo	18	0,9577
pfp_999	$\text{TPR} \geq 0,999$, FP progresivo	21	0,9841
pfp_999_all_data	$\text{TPR} \geq 0,999$, FP progresivo, todos los datos	21	0,9956

Tabla 5.1: Configuraciones experimentales y resultados. Todos los modelos alcanzan FP = 0 % en test.

5.1.1. Filtros Haar por etapa

La Tabla 5.2 muestra el número de filtros Haar (*stumps*) seleccionados por AdaBoost en cada etapa para las cuatro configuraciones base. Como vemos, los modelos más exigentes ($\text{TPR} \geq 0,999$) requieren significativamente más filtros por etapa y en total: `balanced_999` usa 4 601 filtros frente a los 1 382 de `balanced_99`. La tasa progresiva de FP tiene un efecto similar, ya que al exigir menores tasas de FP en las primeras etapas, cada una necesita más filtros para alcanzar el objetivo.

Modelo	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24	S25	S26	S27	Total
99	2	5	5	8	6	12	15	23	26	36	37	40	54	62	67	77	90	93	123	125	141	157	178	-	-	-	-	-	1382
999	4	6	8	11	15	24	30	38	44	56	70	76	93	112	128	141	157	187	202	222	256	284	330	346	367	419	475	500	4601
pfp_99	4	10	13	22	39	36	47	56	68	87	84	91	98	112	133	137	159	178	-	-	-	-	-	-	-	-	-	-	1374
pfp_999	7	23	38	59	86	85	109	133	155	181	186	218	225	246	302	317	355	395	414	452	500	-	-	-	-	-	-	-	4486
pfp_999_all	11	34	52	85	136	153	188	250	302	371	370	443	500	500	500	500	500	500	500	500	500	-	-	-	-	-	-	-	6895

Tabla 5.2: Número de filtros Haar seleccionados por etapa en cada configuración.

En el caso de `pfp_999_all`, se observa que 9 de sus 21 etapas alcanzan el límite máximo impuesto de 500 *stumps* sin llegar al objetivo de FP del 50 %, manteniendo una $\text{TPR} \geq 0,999$. Esto indica que, con la mayor diversidad de muestras del conjunto completo, el modelo necesita más capacidad por etapa de la que el límite permite, y que aumentar este tope podría mejorar la convergencia y reducir el número total de etapas.

5.1.2. Filtros de la primera etapa

La Figura 5.2 muestra los filtros Haar seleccionados en la primera etapa para cada configuración. Los 2–3 primeros filtros son prácticamente idénticos en todas las configuraciones, ya que corresponden a las características más discriminativas de una cara (típicamente filtros horizontales sobre la zona de ojos y nariz). Sin embargo, a partir del cuarto filtro (en las configuraciones que lo tienen), las selecciones divergen significativamente según la exigencia de la configuración.

Cabe destacar que el nuevo tipo de característica Haar propuesto (cuadrado con uno concéntrico interior) apenas ha sido seleccionado por AdaBoost en ninguna de las configuraciones, lo que indica que no aporta una mejora discriminativa significativa respecto a los filtros Haar clásicos.

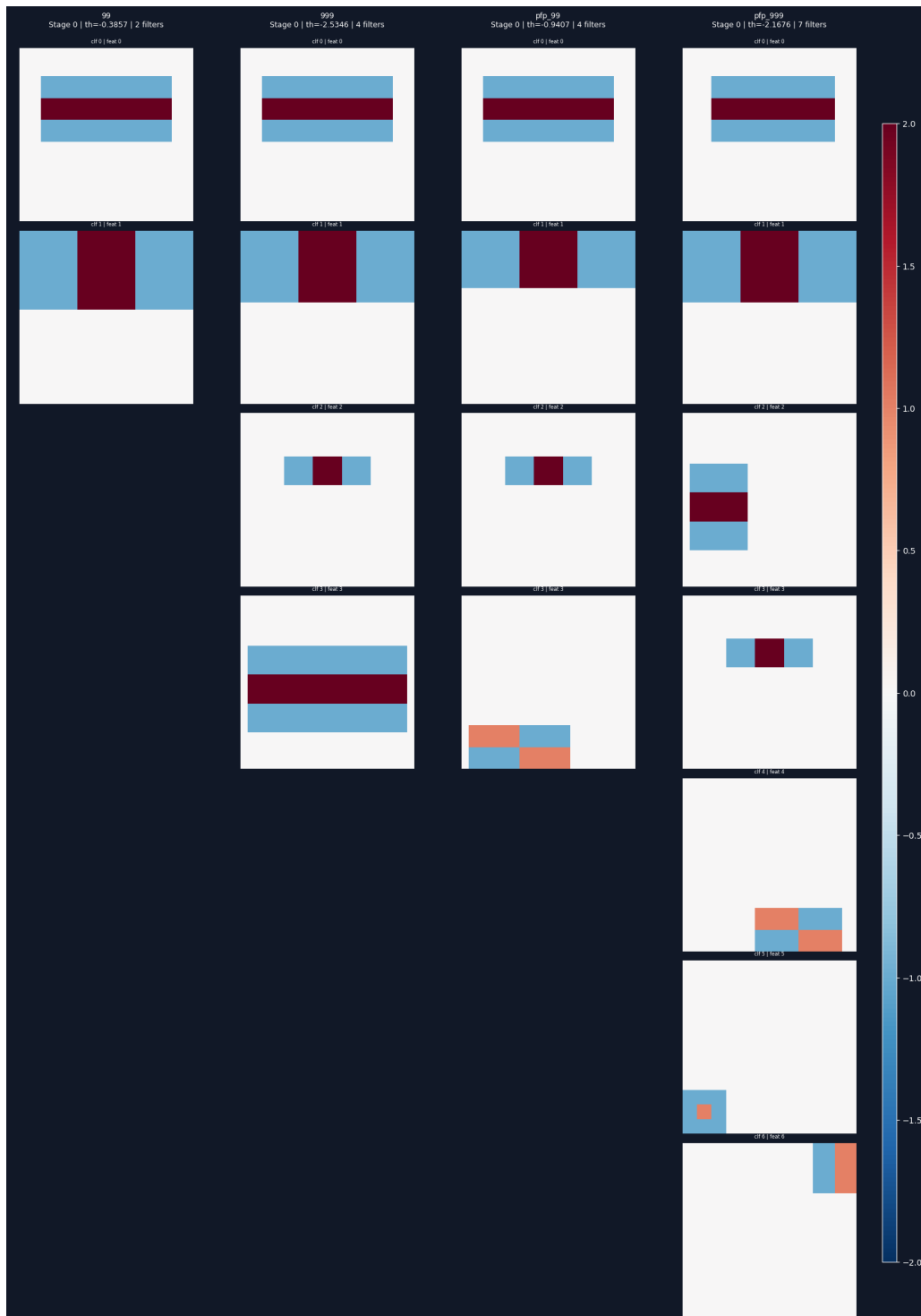


Figura 5.2: Filtros Haar seleccionados en la primera etapa para cada configuración. En orden de izquierda a derecha: nopfp_99, nopfp_999, pfp_99, pfp_999 y pfp_999_all_data.

5.1.3. Inferencia en tiempo real

Para evaluar el rendimiento práctico de los modelos, se ha medido la velocidad de inferencia y la calidad de detección en tiempo real utilizando la cámara del equipo. Las pruebas consistieron en grabaciones de aproximadamente un minuto en las que se mostraban imágenes, personas y movimiento, obteniendo mediciones muy estables.

Modelo	FPS	Observaciones
balanced_99	~72,3	Menor precisión
balanced_999	~60,5	Detección robusta
pfp_99	~69,0	Menor precisión
pfp_999	~59,3	Detección robusta
pfp_999_all	~53,1	Detección robusta

Tabla 5.3: Velocidad de inferencia en tiempo real para cada configuración.

Todos los modelos superan con holgura el umbral de tiempo real (30 FPS), pero la diferencia de velocidad entre familias es apreciable: los modelos con $\text{TPR} \geq 0,99$ alcanzan ~70 FPS, mientras que los de $\text{TPR} \geq 0,999$ se sitúan en torno a 60 FPS, y el modelo entrenado con todos los datos baja a ~53 FPS. Aunque significativa en términos relativos, esta diferencia no compromete la operación en tiempo real en ningún caso.

Un resultado relevante es que la tasa progresiva de FP no aporta una ventaja clara ni en velocidad ni en precisión. En velocidad, los modelos con tasa progresiva son incluso ligeramente más lentos que sus equivalentes sin ella (pfp_99 a 69,0 FPS frente a balanced_99 a 72,3; pfp_999 a 59,3 frente a balanced_999 a 60,5). Esto se explica porque, aunque la tasa progresiva reduce el número de etapas, concentra más filtros en cada una, resultando en un número total de filtros prácticamente idéntico pero en etapas anteriores que han de procesar más crops.

En cambio, la diferencia en calidad de detección entre las dos familias de TPR sí es notable: 0,9577 frente a 0,9516 para la familia 0,99, y 0,9841 frente a 0,9833 para la 0,999. En la práctica, los modelos con $\text{TPR} \geq 0,99$ presentan fallos frecuentes en casos como personas con gafas, que a menudo no son reconocidas correctamente. Los modelos con $\text{TPR} \geq 0,999$, al haber sido entrenados con un requisito más estricto en cada etapa, no sufren este problema en la misma medida y tienden a seguir la cara de forma más consistente incluso cuando se gira lateralmente o se rota. En definitiva, la penalización en velocidad de los modelos más exigentes (~10 FPS menos) resulta un precio bajo a cambio de una mejora cualitativa sustancial en robustez.

5.2 Conclusiones

Tras la implementación completa del detector facial Viola-Jones y los cinco experimentos realizados, se extraen las siguientes conclusiones:

- **$\text{TPR} \geq 0,999$ frente a 0,99:** La elección de la TPR mínima por etapa es el factor determinante del rendimiento. Los modelos con 0,999 son claramente superiores en calidad de detección (mejor seguimiento de caras rotadas, menor sensibilidad a gafas), con una penalización en velocidad moderada (~60 FPS frente a ~70 FPS) que no compromete la operación en tiempo real. El modelo final con todos los datos alcanza ~53 FPS y la TPR final más alta de 0,9891, aunque en su caso la TPR desciende de forma más lenta a lo largo de las etapas al haber sido entrenado con una mayor variedad de muestras, lo que sugiere que aún podría seguir mejorando con más etapas.
- **Tasa progresiva de FP:** Contrariamente a lo esperado, la tasa progresiva de FP no ha aportado beneficios claros. No mejora la velocidad de inferencia (los modelos progresivos son incluso ligeramente más lentos), ya que aunque reduce el número de etapas, concentra más filtros en cada una, manteniendo un total de filtros prácticamente idéntico. La mejora en TPR es marginal ($<0,01$). Su única ventaja práctica ha sido simplificar el proceso de entrenamiento al requerir menos rondas de *hard mining*, motivo por el cual se seleccionó para entrenar el modelo final con todos los datos.

- **Escalado con datos:** El modelo pfp_999_all, entrenado con todas las muestras disponibles, es el más lento (~ 53 FPS) y el que más filtros usa (6895 frente a 4486), con 9 de sus 21 etapas alcanzando el máximo de 500 *stumps*. Sin embargo, es también el que mejor clasifica, con la TPR final más alta (0,9891). Su TPR desciende más gradualmente etapa a etapa que en el resto de configuraciones, indicando que la mayor diversidad de datos dificulta la convergencia pero produce un clasificador más robusto al final de la cascada. Aumentar el límite de *stumps* por etapa o añadir más etapas podría mejorar aún más el modelo.
- **Filtros Haar:** Los primeros filtros seleccionados por AdaBoost son consistentes entre todas las configuraciones, lo que confirma que ciertas características faciales (contraste ojos-frente, nariz-mejillas) son universalmente discriminativas. El tipo de filtro propuesto (cuadrado con concéntrico interior) apenas ha sido seleccionado, lo que indica que no aporta valor adicional frente a los filtros clásicos.
- **Optimizaciones:** La paralelización del entrenamiento, la vectorización de la inferencia con numba y la estrategia de *hard negative mining* con preservación de falsos positivos han sido fundamentales para hacer viable el entrenamiento y alcanzar detección en tiempo real en todas las configuraciones.